SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **N. HEMA SREE (192511100)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **N. HEMA SREE**

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

ANSWER:

Problem Understanding

Given:

- Total data received = **8000 bits**
- Time = **4 seconds**
- Packets sent = **200**
- Packets received = **180**

Formulas:

Throughput = Total Data Received / Time

Packet Loss % = ((Packets Sent − Packets Received) / Packets Sent) × 100

The network is **Efficient** when:

- Throughput > 1500 bps
- Packet Loss < 10%

Otherwise, it is **Inefficient**.

Pseudocode

```
ALGORITHM CalculateNetworkPerformance

BEGIN
   // Input values
   totalDataReceived ← 8000
   time ← 4
   packetsSent ← 200
   packetsReceived ← 180
   // Validate input
   IF time = 0 THEN
      DISPLAY "Error: Time cannot be zero."
      STOP
   END IF
   IF packetsSent = 0 THEN
      DISPLAY "Error: Total packets sent cannot be zero."
      STOP
   END IF
   IF packetsReceived > packetsSent THEN
      DISPLAY "Error: Packets received cannot exceed packets sent."
      STOP
   END IF
   // Calculate throughput
   throughput ← totalDataReceived / time
   // Calculate packet loss
   packetsLost ← packetsSent - packetsReceived
   packetLoss ← (packetsLost / packetsSent) × 100
   // Display calculated results
   DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
```

```
    DISPLAY "Total Data Received: ", totalDataReceived, " bits"
    DISPLAY "Time: ", time, " seconds"
    DISPLAY "Packets Sent: ", packetsSent
    DISPLAY "Packets Received: ", packetsReceived
    DISPLAY "Packets Lost: ", packetsLost
    DISPLAY "Throughput: ", throughput, " bps"
    DISPLAY "Packet Loss: ", packetLoss, "%"
    // Classify network
    IF throughput > 1500 AND packetLoss < 10 THEN
        networkStatus ← "Efficient"
    ELSE
        networkStatus ← "Inefficient"
    END IF
    DISPLAY "Network Status: ", networkStatus
END
```

Calculation

Throughput:

8000 / 4 = **2000 bps**

Packets Lost:

200 − 180 = **20 packets**

Packet Loss:

(20 / 200) × 100 = **10%**

Since the throughput is greater than 1500 bps, but packet loss is **not less than 10%**, the network is:

Network Status = Inefficient

Final Output

Total Data Received: 8000 bits

Time: 4 seconds

Packets Sent: 200

Packets Received: 180

Packets Lost: 20

Throughput: 2000 bps

Packet Loss: 10%

Network Status: Inefficient

Why this is good: The algorithm validates zero values, performs all required calculations, uses conditional logic, and produces a complete performance summary.

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

ANSWER:

Problem Understanding

The organization has multiple network links. The algorithm should:

1. Periodically collect bandwidth utilization.
2. Store utilization values in an array.
3. Use loops to monitor every link.
4. Check whether utilization exceeds **80%**.
5. Recommend switching traffic to a backup link when overloaded.
6. Continue monitoring periodically.

Pseudocode

ALGORITHM MonitorBandwidth

BEGIN

```
// Number of network links
numberOfLinks ← 5
// Array to store bandwidth utilization
DECLARE utilization[5]
// Monitoring interval
monitoringInterval ← 60 seconds
WHILE TRUE DO
   DISPLAY "----- BANDWIDTH MONITORING -----"
   // Collect utilization of each link
   FOR i ← 1 TO numberOfLinks DO
      utilization[i] ← GET_BANDWIDTH_UTILIZATION(i)
      DISPLAY "Link ", i, " Utilization: ",
            utilization[i], "%"
      // Check for overloaded link
      IF utilization[i] > 80 THEN
         DISPLAY "WARNING: Link ", i,
               " is overloaded."
         DISPLAY "Recommendation: Switch traffic "
               "to backup link."
         SWITCH_TRAFFIC_TO_BACKUP(i)
      ELSE
         DISPLAY "Link ", i,
               " is operating normally."
      END IF
   END FOR
   // Wait before next monitoring cycle
   WAIT monitoringInterval
```

END WHILE

END

Explanation

The algorithm uses an **array** called utilization[] to store the bandwidth usage of every network link.

The FOR loop checks each link one by one. If the utilization is greater than **80%**, the link is considered overloaded. The algorithm then generates a warning and recommends or performs traffic switching to a backup link.

The WHILE TRUE loop ensures that monitoring continues periodically.

How it supports bandwidth management

- **Continuous monitoring:** Network links are checked repeatedly.
- **Early congestion detection:** Links exceeding 80% utilization are detected before severe congestion occurs.
- **Backup traffic routing:** Traffic can be moved to another link.
- **Better resource utilization:** Available bandwidth is used efficiently.
- **Reduced packet loss:** Overloaded links can be avoided.
- **Improved network performance:** Congestion and delays are reduced.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails,

automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

ANSWER:

Best Communication Path Selection

Problem Understanding

There are six branch offices connected using multiple network links. The best path should not be selected only based on distance.

The algorithm considers:

- **Bandwidth** – higher bandwidth is better.
- **Delay** – lower delay is better.
- **Utilization** – lower utilization is better.
- **Link status** – failed links cannot be selected.

A link quality score can be calculated as:

Quality Score = (0.5 × Normalized Bandwidth) + (0.3 × Normalized Delay) + (0.2 × Normalized Utilization)

For delay and utilization, lower values are better, so their normalized values can be calculated as:

Delay Score = 1 − (Delay / Maximum Delay)

Utilization Score = 1 − (Utilization / 100)

The path with the **highest total quality score** is selected.

Pseudocode

```
ALGORITHM SelectBestNetworkPath

BEGIN
   numberOfBranches ← 6
   monitoringInterval ← 60 seconds
```

```
WHILE TRUE DO
    DISPLAY "----- NETWORK PATH MONITORING -----"
    // Collect information about every network link
    FOR each link L in NetworkLinks DO
        bandwidth[L] ← GET_BANDWIDTH(L)
        delay[L] ← GET_DELAY(L)
        utilization[L] ← GET_UTILIZATION(L)
        status[L] ← GET_LINK_STATUS(L)
    END FOR
    // Check every branch
    FOR branch ← 1 TO numberOfBranches DO
        bestScore ← -1
        bestPath ← NONE
        // Find all possible paths to the branch
        paths ← FIND_ALL_PATHS(Headquarters, branch)
        FOR each path P in paths DO
            pathAvailable ← TRUE
            totalBandwidthScore ← 0
            totalDelayScore ← 0
            totalUtilizationScore ← 0
            // Examine every link in the path
            FOR each link L in P DO
                // Check whether link has failed
                IF status[L] = "FAILED" THEN
                    pathAvailable ← FALSE
                END IF
                // Check whether link is overloaded
                IF utilization[L] > 80 THEN
                    pathAvailable ← FALSE
```

```
    END IF
    // Calculate normalized metrics
    bandwidthScore ← bandwidth[L] /
                    MAX_BANDWIDTH
    delayScore ← 1 -
                (delay[L] / MAX_DELAY)
    utilizationScore ← 1 -
                    (utilization[L] / 100)
    // Add scores
    totalBandwidthScore ←
        totalBandwidthScore + bandwidthScore
    totalDelayScore ←
        totalDelayScore + delayScore
    totalUtilizationScore ←
        totalUtilizationScore + utilizationScore
END FOR
// Select only available paths
IF pathAvailable = TRUE THEN
    // Calculate overall path quality
    qualityScore ←
        (0.5 × totalBandwidthScore) +
        (0.3 × totalDelayScore) +
        (0.2 × totalUtilizationScore)
    // Select highest quality path
    IF qualityScore > bestScore THEN
        bestScore ← qualityScore
        bestPath ← P
    END IF
```

```
        END IF
    END FOR
    // Check whether a path is available
    IF bestPath ≠ NONE THEN
        SET_ROUTE(branch, bestPath)
        DISPLAY "Branch ", branch,
                ": Best Path = ", bestPath
        DISPLAY "Quality Score = ", bestScore
    ELSE
        DISPLAY "WARNING: No available path for Branch ",
                branch
        GENERATE_ALERT("No available network path")
    END IF
END FOR
// Monitor overloaded or failed links
FOR each link L in NetworkLinks DO
    IF status[L] = "FAILED" THEN
        GENERATE_ALERT("Link failure detected")
        DISPLAY "Link ", L,
                " has failed. Alternate paths selected."
    ELSE IF utilization[L] > 80 THEN
        GENERATE_ALERT("Link overloaded")
        DISPLAY "Link ", L,
                " is overloaded. Traffic redirected."
    END IF
END FOR
// Wait before next monitoring cycle
WAIT monitoringInterval
```

END WHILE

END

Explanation of the Algorithm

The algorithm continuously monitors all network links and stores their:

- Bandwidth
- Delay
- Utilization
- Status

For every branch, it identifies possible paths from the headquarters and calculates a **quality score** for each path.

A higher bandwidth improves the score, while high delay and high utilization reduce the score.

The path having the **highest quality score** is selected as the best communication path.

Fault and Overload Handling

If:

utilization > 80%

the link is considered overloaded.

If:

status = FAILED

the link is considered unavailable.

In either case, the algorithm avoids that link and selects the best available alternate path. An alert is also generated for the administrator.